Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Разработка, тестирование и отладка программы СУККТиПС

Выполнил студент: Моисеенко А.Д.
Группа: 551051
Зачетная книжка № 89105027

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 ОПИСАНИЕ СТРУКТУР ДАННЫХ 4
1.1 Структура «Тип комплектующего» 4
1.2 Структура «Комплектующее» 4
1.3 Структура «Совместимость» 5
1.4 Динамические списки 5
2 ОПИСАНИЕ МОДУЛЕЙ ПРОГРАММЫ 6
2.1 Модуль common 6
2.2 Модуль list 6
2.3 Модуль file_io 7
2.4 Модуль ui 7
2.5 Модуль special 7
2.6 Модуль tests 8
2.7 Модуль main 8
3 ТЕСТИРОВАНИЕ ПРОГРАММЫ 9
3.1 Модульные тесты 9
3.2 Ручные тест-кейсы 11
4 ТЕКСТ ПРОГРАММЫ С КОММЕНТАРИЯМИ 12
4.1 Файл common.h 12
4.2 Файл common.cpp 13
4.3 Файл list.h 15
4.4 Файл list.cpp 15
4.5 Файл file_io.h 18
4.6 Файл file_io.cpp 18
4.7 Файл ui.h 20
4.8 Файл ui.cpp 21
4.9 Файл special.h 29
4.10 Файл special.cpp 29
4.11 Файл tests.h 33
4.12 Файл tests.cpp 33
4.13 Файл main.cpp 36
5 GIT-ЛОГИ 37
ЗАКЛЮЧЕНИЕ 38

ВВЕДЕНИЕ

Целью учебной практики является разработка программного продукта — системы учёта комплектующих персональных компьютеров и подбора сборок (СУККТиПС). Программа предназначена для автоматизации ведения учёта комплектующих компьютерной техники, проверки совместимости комплектующих между собой и подбора оптимального варианта сборки ПК в заданном ценовом диапазоне.

Объектом автоматизации являются процессы ручного ведения списков комплектующих, типов комплектующих и их совместимости, а также процедура подбора готовой сборки под бюджет клиента.

Программный продукт реализован на языке C++ (стандарт C++17) с использованием ручного управления памятью (операторы `new` и `delete`), динамических односвязных списков и типизированных бинарных файлов для хранения данных.

В ходе работы решены следующие задачи:

- разработано техническое задание на систему;
- спроектирована структура данных и модульная архитектура приложения;
- реализованы все десять пунктов пользовательского меню;
- реализованы две специальные функции (подбор вариантов сборки и оформление заказа);
- выполнено тестирование — как модульными тестами, так и вручную;
- оформлена техническая документация.

1 ОПИСАНИЕ СТРУКТУР ДАННЫХ

В программе используются три основные структуры данных, каждая из которых существует в двух видах: как POD-структура фиксированного размера (для чтения и записи в типизированный бинарный файл) и как узел динамического односвязного списка (для работы в оперативной памяти).

1.1 Структура «Тип комплектующего»

Описывает одну категорию комплектующих (например, «Процессор», «Материнская плата»). Поля структуры приведены в таблице 1.

Таблица 1 – Поля структуры «Тип комплектующего»

Поле	Тип	Размер	Назначение
code	int	4 байта	Уникальный код типа
name	char[64]	64 байта	Название типа (до 63 символов + '\0')
next	Type*	8 байт	Указатель на следующий узел (только в динамическом списке)

1.2 Структура «Комплектующее»

Описывает конкретное комплектующее с указанием его типа, фирмы-изготовителя, модели, параметров, цены и наличия (таблица 2).

Таблица 2 – Поля структуры «Комплектующее»

Поле	Тип	Размер	Назначение
code	int	4 байта	Уникальный код комплектующего
typeCode	int	4 байта	Ссылка на код типа
manufacturer	char[64]	64 байта	Фирма-изготовитель
model	char[64]	64 байта	Модель
params	char[128]	128 байт	Параметры (характеристики)
price	double	8 байт	Цена в условных единицах
available	int	4 байта	Количество единиц в наличии
next	Component*	8 байт	Указатель на следующий узел

1.3 Структура «Совместимость»

Описывает факт совместимости двух комплектующих (таблица 3). Хранение пар симметрично: наличие записи (a, b) означает, что a совместимо с b и b совместимо с a.

Таблица 3 – Поля структуры «Совместимость»

Поле	Тип	Размер	Назначение
compCode	int	4 байта	Код первого комплектующего
compatCode	int	4 байта	Код второго (совместимого) комплектующего
next	Compat*	8 байт	Указатель на следующий узел

1.4 Динамические списки

Все три структуры хранятся в оперативной памяти в виде односвязных линейных списков. Каждый узел выделяется оператором `new` при добавлении и освобождается `delete` при удалении или при завершении работы.

Память освобождается централизованно функцией `clearAll()`, которая последовательно проходит по всем трём спискам и удаляет их узлы.

2 ОПИСАНИЕ МОДУЛЕЙ ПРОГРАММЫ

Приложение разделено на семь модулей, каждый из которых решает свою задачу. Такое разделение облегчает сопровождение, тестирование и повторное использование кода.

2.1 Модуль common

Содержит общие типы данных, глобальные указатели на начала списков, имена файлов данных и утилиты. Функции модуля:

- `safeCopy()` — безопасное копирование строки с обрезкой до заданной длины и установкой нуль-терминатора;
- `fileExists()` — проверка существования файла;
- `inputInt()`, `inputDouble()`, `inputString()` — безопасный ввод целых, вещественных чисел и строк с проверкой корректности;
- `utf8Length()` — подсчёт длины строки в символах UTF-8 (для корректного выравнивания кириллицы в таблицах);
- `padRight()`, `padLeft()` — выравнивание строк с учётом UTF-8;
- `fmtPrice()` — форматирование цены с двумя знаками после запятой;
- `clearScreen()`, `pauseScreen()` — очистка экрана и пауза.

2.2 Модуль list

Реализует работу с динамическими односвязными списками. Функции модуля:

- `addTypeNode()`, `addComponentNode()`, `addCompatNode()` — добавление узла в конец соответствующего списка;
- `clearTypes()`, `clearComponents()`, `clearCompats()`, `clearAll()` — освобождение памяти, занимаемой списками;
- `typeExists()`, `componentExists()` — проверка существования элемента по коду;
- `areCompatible()` — проверка совместимости двух комплектующих (симметричная);
- `removeCompatByComponentCode()` — каскадное удаление всех записей совместимости при удалении комплектующего;
- `sortTypes()`, `sortComponents()`, `sortCompats()` — сортировки по заданным критериям методом пузырька с обменом данных узлов (указатели не переставляются).

2.3 Модуль file_io

Отвечает за загрузку и сохранение данных в типизированных бинарных файлах. Функции модуля:

- `readTypesFromFile()`, `readComponentsFromFile()`, `readCompatsFromFile()` — чтение POD-структур из файлов в динамические списки;
- `saveAllFiles()` — сохранение текущего состояния всех трёх списков в бинарные файлы;
- `createSampleFiles()` — генерация демонстрационного набора данных при первом запуске программы.

2.4 Модуль ui

Реализует пользовательский интерфейс: меню, подменю, обработку ввода, вывод таблиц. Функции модуля:

- `mainMenu()` — главное меню с пунктами 1–10;
- `menuRead()` — пункт 1 (чтение данных);
- `menuView()` — пункт 2 (просмотр списков);
- `menuSort()` — пункт 3 (сортировка);
- `menuSearch()` — пункт 4 (поиск);
- `menuAdd()` — пункт 5 (добавление);
- `menuDelete()` — пункт 6 (удаление);
- `menuEdit()` — пункт 7 (редактирование);
- `printTypes()`, `printComponents()`, `printCompats()` — вывод таблиц на экран.

2.5 Модуль special

Реализует специальные функции (пункт 8 меню). Функции модуля:

- `buildVariants()` — перебор всех возможных вариантов сборки ПК в заданном ценовом диапазоне методом поиска в глубину (DFS) с проверкой попарной совместимости комплектующих. Результат выводится на экран и сохраняется в файл `sf_builds.txt`;
- `placeOrder()` — оформление заказа выбранного варианта: проверка наличия, списание со склада, запись в файл `sf_order.txt`;
- `menuSpecial()` — подменю СФ.

2.6 Модуль tests

Содержит модульные тесты (собственный мини-фреймворк на основе макроса `CHECK`). Тестируются:

- работа с UTF-8 (`utf8Length`, `padLeft`, `padRight`);
- добавление, поиск, удаление элементов списков;
- проверка совместимости;
- сортировки по всем критериям;
- цикл «сохранение → загрузка» (round-trip) файлов;
- базовая логика подбора вариантов.

2.7 Модуль main

Точка входа в программу. Устанавливает кодировку UTF-8 в консоли Windows (при необходимости), проверяет наличие флага `--test` и либо запускает тесты, либо передаёт управление в `mainMenu()`.

3 ТЕСТИРОВАНИЕ ПРОГРАММЫ

3.1 Модульные тесты

Модульные тесты запускаются командой `./app --test` (или `make test`). Все тесты используют макрос `CHECK(условие, сообщение)` и выводят результат вида `[ OK ]` или `[FAIL]`. Перечень тест-кейсов приведён в таблице 4.

Таблица 4 – Модульные тест-кейсы

№	Проверка	Ожидаемый результат
1	utf8Length("abc")	3
2	utf8Length("привет")	6
3	padRight("аб", 5) — длина в символах	5
4	padLeft("аб", 5) — длина в символах	5
5	addTypeNode — порядок сохранён	types->code == 1, types->next->code == 2
6	typeExists(1), typeExists(3)	true, false
7	componentExists(10)	true
8	areCompatible(10, 20) и (20, 10)	true и true (симметрия)
9	areCompatible(10, 10)	true (сам с собой)
10	areCompatible(10, 99)	false
11	removeCompatByComponentCode(20)	areCompatible(10, 20) == false
12	clearAll()	все указатели == nullptr
13	sortComponents по цене ↑	100, 200, 300
14	sortComponents по цене ↓	300 первый
15	sortComponents по коду	1, 2, 3
16	sortTypes по коду	1, 3, 5
17	sortTypes по названию	A, C, E
18	Round-trip файла types.bin	код и название совпадают
19	Round-trip файла components.bin	все поля совпадают
20	Round-trip файла compat.bin	пара (201, 201) сохранена
21	Логика СФ: минимум один вариант	found >= 1

Результаты тестирования представлены на рисунке 1.

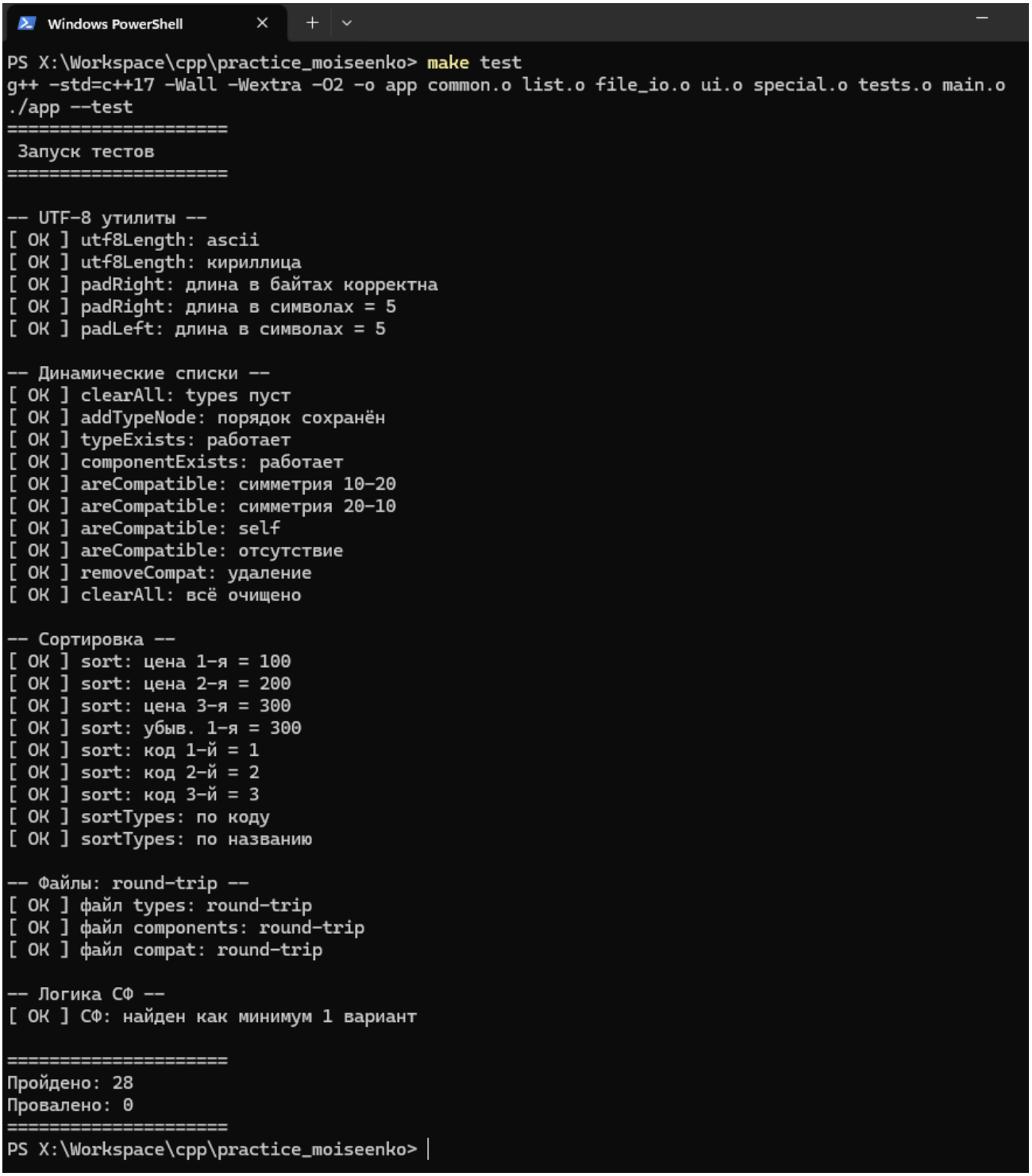


Рисунок 1 – Скриншот пройденных тестов

3.2 Ручные тест-кейсы

Помимо модульных тестов, проверка работы проводилась вручную по сценариям, приведённым в таблице 5.

Таблица 5 – Ручные тест-кейсы

№	Действия	Ожидаемый результат
1	Запуск программы → пункт 1	Файлы не найдены → создаются демо-данные → сообщение «Данные загружены в динамические списки»
2	Пункт 2 → 2	Таблица комплектующих с выровненными колонками (кириллица не «плывёт»)
3	Пункт 3 → 4 (цена ↑)	Сообщение «Отсортировано». Проверка через п. 2 — цены по возрастанию
4	Пункт 4 → 4: код типа = 1, код X = 101	Выводятся комплектующие типа 1, совместимые с 101 (например, материнская плата ASUS Prime B660)
5	Пункт 4 → 4: код типа = 1, код X = 101, затем ищем тип 1, совместимый с 102 (AMD)	Ничего не найдено (AMD несовместим с LGA1700-платой)
6	Пункт 5 → 2: код = 999, тип = 1	«Комплектующее добавлено»
7	Пункт 5 → 2: код = 101 (занят)	«Такой код уже существует»
8	Пункт 6 → 2: код = 999	«Комплектующее удалено, связки совместимости очищены»
9	Пункт 7 → 2: код = 201, изменить цену на 14000	«Комплектующее отредактировано». Проверка через п. 2
10	Пункт 8 → 1: диапазон 40000..70000	Список вариантов, отсортированных по цене возрастанию, + файл `sf_builds.txt`
11	Пункт 8 → 2: заказ варианта №1, 2 шт.	Заказ оформлен, наличие уменьшено на 2, файл `sf_order.txt` дополнен
12	Пункт 8 → 2: заказать 100 шт.	«Недостаточно комплектующего …»
13	Пункт 9 → выход без сохранения → повторный запуск	Изменения, сделанные в п. 6–7, не сохранились
14	Пункт 10 → выход с сохранением → повторный запуск → п. 1	Все изменения на месте
15	Ввод буквы в числовое поле	«Ошибка ввода. Повторите:» и повторный запрос

4 ТЕКСТ ПРОГРАММЫ С КОММЕНТАРИЯМИ

4.1 Файл common.h

```
#pragma once
#include <iostream>
#include <fstream>
#include <iomanip>
#include <sstream>
#include <cstring>
#include <limits>
#include <vector>
#include <algorithm>
#include <string>
#include <functional>

// ---------- имена файлов ----------
extern const char* FILE_TYPES;        // типы комплектующих
extern const char* FILE_COMPONENTS;   // комплектующие
extern const char* FILE_COMPAT;       // совместимость

// ---------- POD-структуры для файлов ----------
struct TypeRec      { int code; char name[64]; };

struct ComponentRec {
    int code; int typeCode;
    char manufacturer[64];
    char model[64];
    char params[128];
    double price;
    int available;
};

struct CompatRec    { int compCode; int compatCode; };

// ---------- узлы динамических списков ----------
struct Type {
    int code;
    char name[64];
    Type* next;
};

struct Component {
    int code; int typeCode;
    char manufacturer[64];
    char model[64];
    char params[128];
    double price; int available;
    Component* next;
};

struct Compat {
    int compCode; int compatCode;
    Compat* next;
};

// ---------- глобальные указатели ----------
```

```
extern Type*      types;
extern Component* components;
extern Compat*    compats;

// ---------- утилиты ----------
void safeCopy(char* dst, const char* src, int size);
void safeCopy(char* dst, const std::string& src, int size);
bool fileExists(const char* name);

int    inputInt(const std::string& prompt);
double inputDouble(const std::string& prompt);
void   inputString(const std::string& prompt, char* dest, int size);

// подсчёт длины строки в символах UTF-8 и выравнивание
size_t      utf8Length(const std::string& s);
std::string padRight(const std::string& s, int width);
std::string padLeft (const std::string& s, int width);
std::string fmtPrice(double v);

void clearScreen();
void pauseScreen();
```

4.2 Файл common.cpp

```
#include "common.h"

const char* FILE_TYPES      = "types.bin";
const char* FILE_COMPONENTS = "components.bin";
const char* FILE_COMPAT     = "compat.bin";

Type*      types      = nullptr;
Component* components = nullptr;
Compat*    compats    = nullptr;

// Безопасное копирование: обрезаем до size-1 и ставим '\0'
void safeCopy(char* dst, const char* src, int size) {
    if (size <= 0) return;
    std::strncpy(dst, src, size - 1);
    dst[size - 1] = '\0';
}
void safeCopy(char* dst, const std::string& src, int size) {
    safeCopy(dst, src.c_str(), size);
}

bool fileExists(const char* name) {
    std::ifstream f(name, std::ios::binary);
    return f.good();
}

// Ввод целого с защитой от некорректного ввода
int inputInt(const std::string& prompt) {
    std::cout << prompt;
    int x;
    while (!(std::cin >> x)) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Ошибка ввода. " << prompt;
```

```
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return x;
}

double inputDouble(const std::string& prompt) {
    std::cout << prompt;
    double x;
    while (!(std::cin >> x)) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Ошибка ввода. " << prompt;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return x;
}

void inputString(const std::string& prompt, char* dest, int size) {
    std::cout << prompt;
    std::string s;
    std::getline(std::cin, s);
    safeCopy(dest, s, size);
}

// Считаем символы UTF-8 (кириллица занимает 2 байта, но 1 символ)
size_t utf8Length(const std::string& s) {
    size_t len = 0;
    for (unsigned char c : s)
        if ((c & 0xC0) != 0x80) ++len;
    return len;
}

std::string padRight(const std::string& s, int width) {
    size_t len = utf8Length(s);
    if (len >= (size_t)width) return s;
    return s + std::string(width - len, ' ');
}
std::string padLeft(const std::string& s, int width) {
    size_t len = utf8Length(s);
    if (len >= (size_t)width) return s;
    return std::string(width - len, ' ') + s;
}
std::string fmtPrice(double v) {
    std::ostringstream os;
    os << std::fixed << std::setprecision(2) << v;
    return os.str();
}

void clearScreen() {
#ifdef _WIN32
    std::system("cls");
#else
    std::system("clear");
#endif
}
void pauseScreen() {
    std::cout << "\nНажмите Enter для продолжения...";
    std::string dummy;
```

```
    std::getline(std::cin, dummy);
}
```

4.3 Файл list.h

```
#pragma once
#include "common.h"

void clearTypes();
void clearComponents();
void clearCompats();
void clearAll();

void addTypeNode(int code, const char* name);
void addComponentNode(int code, int typeCode, const char* man, const char* model,
                      const char* params, double price, int available);
void addCompatNode(int compCode, int compatCode);

bool typeExists(int code);
bool componentExists(int code);
bool areCompatible(int a, int b);

void removeCompatByComponentCode(int code);

void sortTypes(int criterion);
void sortComponents(int criterion);
void sortCompats(int criterion);
```

4.4 Файл list.cpp

```
#include "list.h"
#include <cstring>

void clearTypes()      { while (types)      { Type* p = types;           types =
types->next;           delete p; } }
void clearComponents() { while (components) { Component* p = components; components
= components->next; delete p; } }
void clearCompats()    { while (compats)    { Compat* p = compats;       compats
= compats->next;       delete p; } }
void clearAll()        { clearTypes(); clearComponents(); clearCompats(); }

void addTypeNode(int code, const char* name) {
    Type* n = new Type;
    n->code = code;
    safeCopy(n->name, name, 64);
    n->next = nullptr;
    if (!types) { types = n; return; }
    Type* p = types;
    while (p->next) p = p->next;
    p->next = n;
}

void addComponentNode(int code, int typeCode, const char* man, const char* model,
                      const char* params, double price, int available) {
    Component* n = new Component;
    n->code = code; n->typeCode = typeCode;
```

```
    safeCopy(n->manufacturer, man, 64);
    safeCopy(n->model, model, 64);
    safeCopy(n->params, params, 128);
    n->price = price; n->available = available;
    n->next = nullptr;
    if (!components) { components = n; return; }
    Component* p = components;
    while (p->next) p = p->next;
    p->next = n;
}

void addCompatNode(int compCode, int compatCode) {
    Compat* n = new Compat;
    n->compCode = compCode;
    n->compatCode = compatCode;
    n->next = nullptr;
    if (!compats) { compats = n; return; }
    Compat* p = compats;
    while (p->next) p = p->next;
    p->next = n;
}

bool typeExists(int code) {
    for (Type* p = types; p; p = p->next)
        if (p->code == code) return true;
    return false;
}
bool componentExists(int code) {
    for (Component* p = components; p; p = p->next)
        if (p->code == code) return true;
    return false;
}

// Симметричная проверка совместимости двух комплектующих
bool areCompatible(int a, int b) {
    if (a == b) return true;
    for (Compat* p = compats; p; p = p->next) {
        if ((p->compCode == a && p->compatCode == b) ||
            (p->compCode == b && p->compatCode == a)) return true;
    }
    return false;
}

// Каскадное удаление всех записей совместимости для комплектующего
void removeCompatByComponentCode(int code) {
    while (compats && (compats->compCode == code || compats->compatCode == code))
{
        Compat* p = compats; compats = compats->next; delete p;
    }
    Compat* p = compats;
    while (p && p->next) {
        if (p->next->compCode == code || p->next->compatCode == code) {
            Compat* d = p->next; p->next = d->next; delete d;
        } else p = p->next;
    }
}

// Обмен данными двух узлов (переставляем содержимое, а не указатели)
```

```
static void swapTypeData(Type* a, Type* b) {
    std::swap(a->code, b->code);
    char t[64]; std::strcpy(t, a->name);
    std::strcpy(a->name, b->name); std::strcpy(b->name, t);
}
static void swapComponentData(Component* a, Component* b) {
    std::swap(a->code, b->code); std::swap(a->typeCode, b->typeCode);
    char t[128];
    std::strcpy(t,     a->manufacturer);     std::strcpy(a->manufacturer,     b-
>manufacturer); std::strcpy(b->manufacturer, t);
    std::strcpy(t,  a->model);                  std::strcpy(a->model,  b->model);
std::strcpy(b->model, t);
    std::strcpy(t,  a->params);               std::strcpy(a->params,  b->params);
std::strcpy(b->params, t);
    std::swap(a->price, b->price);   std::swap(a->available, b->available);
}
static void swapCompatData(Compat* a, Compat* b) {
    std::swap(a->compCode, b->compCode);
    std::swap(a->compatCode, b->compatCode);
}

// Сортировки методом «пузырька» с обменом данными узлов
void sortTypes(int criterion) {
    for (Type* p = types; p; p = p->next)
        for (Type* q = p->next; q; q = q->next) {
            bool s = false;
            if (criterion == 1 && p->code > q->code) s = true;
            if (criterion == 2 && std::strcmp(p->name, q->name) > 0) s = true;
            if (s) swapTypeData(p, q);
        }
}
void sortComponents(int criterion) {
    for (Component* p = components; p; p = p->next)
        for (Component* q = p->next; q; q = q->next) {
            bool s = false;
            if (criterion == 1 && p->code > q->code) s = true;
            if (criterion == 2 && p->price > q->price) s = true;
            if (criterion == 3 && p->price < q->price) s = true;
            if (criterion == 4 && p->typeCode > q->typeCode) s = true;
            if (criterion == 5 && std::strcmp(p->manufacturer, q->manufacturer) >
0) s = true;
            if (s) swapComponentData(p, q);
        }
}
void sortCompats(int criterion) {
    for (Compat* p = compats; p; p = p->next)
        for (Compat* q = p->next; q; q = q->next) {
            bool s = false;
            if (criterion == 1 && p->compCode > q->compCode) s = true;
            if (criterion == 2 && p->compatCode > q->compatCode) s = true;
            if (s) swapCompatData(p, q);
        }
}
```

4.5 Файл file_io.h

```
#pragma once
#include "common.h"

void readTypesFromFile();
void readComponentsFromFile();
void readCompatsFromFile();
void saveAllFiles();
void createSampleFiles();
```

4.6 Файл file_io.cpp

```
#include "file_io.h"
#include "list.h"
#include <vector>

// Чтение типов из бинарного файла и добавление узлов в список
void readTypesFromFile() {
    std::ifstream in(FILE_TYPES, std::ios::binary);
    if (!in) return;
    TypeRec r;
    while (in.read((char*)&r, sizeof(r)))
        addTypeNode(r.code, r.name);
}

// Чтение комплектующих
void readComponentsFromFile() {
    std::ifstream in(FILE_COMPONENTS, std::ios::binary);
    if (!in) return;
    ComponentRec r;
    while (in.read((char*)&r, sizeof(r)))
        addComponentNode(r.code, r.typeCode, r.manufacturer, r.model,
                         r.params, r.price, r.available);
}

// Чтение записей совместимости
void readCompatsFromFile() {
    std::ifstream in(FILE_COMPAT, std::ios::binary);
    if (!in) return;
    CompatRec r;
    while (in.read((char*)&r, sizeof(r)))
        addCompatNode(r.compCode, r.compatCode);
}

// Сохранение всех трёх списков в типизированные бинарные файлы
void saveAllFiles() {
    {
        std::ofstream t(FILE_TYPES, std::ios::binary);
        for (Type* p = types; p; p = p->next) {
            TypeRec r{}; r.code = p->code;
            safeCopy(r.name, p->name, 64);
            t.write((char*)&r, sizeof(r));
        }
    }
    {
        std::ofstream c(FILE_COMPONENTS, std::ios::binary);
```

```
        for (Component* p = components; p; p = p->next) {
            ComponentRec r{};
            r.code = p->code; r.typeCode = p->typeCode;
            safeCopy(r.manufacturer, p->manufacturer, 64);
            safeCopy(r.model,        p->model,        64);
            safeCopy(r.params,       p->params,       128);
            r.price = p->price; r.available = p->available;
            c.write((char*)&r, sizeof(r));
        }
    }
    {
        std::ofstream k(FILE_COMPAT, std::ios::binary);
        for (Compat* p = compats; p; p = p->next) {
            CompatRec r{}; r.compCode = p->compCode;
            r.compatCode = p->compatCode;
            k.write((char*)&r, sizeof(r));
        }
    }
}

// Демонстрационный набор данных — используется при первом запуске
void createSampleFiles() {
    {
        std::ofstream t(FILE_TYPES, std::ios::binary);
        auto wt = [&](int code, const char* name) {
            TypeRec r{}; r.code = code;
            safeCopy(r.name, name, 64);
            t.write((char*)&r, sizeof(r));
        };
        wt(1, "Процессор");
        wt(2, "Материнская плата");
        wt(3, "Память");
        wt(4, "Видеокарта");
        wt(5, "Блок питания");
        wt(6, "Накопитель");
    }

    std::vector<ComponentRec> comps;
    auto addComp = [&](int code, int tc, const char* m, const char* md,
                       const char* p, double pr, int a) {
        ComponentRec r{};
        r.code = code; r.typeCode = tc;
        safeCopy(r.manufacturer, m, 64);
        safeCopy(r.model,        md, 64);
        safeCopy(r.params,       p, 128);
        r.price = pr; r.available = a;
        comps.push_back(r);
    };
    addComp(101, 1, "Intel",    "Core i5-12400",   "LGA1700 2.5GHz", 18000, 5);
    addComp(102, 1, "AMD",      "Ryzen 5 5600",    "AM4 3.5GHz",     15000, 4);
    addComp(201, 2, "ASUS",     "Prime B660",      "LGA1700 DDR4",   12000, 3);
    addComp(202, 2, "MSI",      "B550-A Pro",      "AM4 DDR4",       10000, 2);
    addComp(301, 3, "Kingston", "Fury Beast 16GB", "DDR4 3200",       5000,10);
    addComp(302, 3, "Samsung",  "DDR4 16GB",       "DDR4 2666",       4500, 8);
    addComp(401, 4, "NVIDIA",   "RTX 3060",        "12GB",           30000, 2);
    addComp(402, 4, "AMD",      "Radeon RX 6600",  "8GB",            25000, 3);
    addComp(501, 5, "Corsair",  "CV650",           "650W",            5500, 5);
    addComp(502, 5, "DeepCool", "PF500",           "500W",            3500, 6);
```

```
    addComp(601, 6, "WD",       "Blue 1TB",        "HDD 7200rpm",     4000, 7);

    {
        std::ofstream c(FILE_COMPONENTS, std::ios::binary);
        for (auto& r : comps) c.write((char*)&r, sizeof(r));
    }

    // Формируем совместимость всех пар разных типов, кроме несовместимых CPU-MB
    {
        std::ofstream k(FILE_COMPAT, std::ios::binary);
        for (size_t i = 0; i < comps.size(); ++i)
            for (size_t j = i + 1; j < comps.size(); ++j) {
                if (comps[i].typeCode == comps[j].typeCode) continue;
                int t1 = comps[i].typeCode, t2 = comps[j].typeCode;
                bool ok = true;
                if ((t1 == 1 && t2 == 2) || (t1 == 2 && t2 == 1)) {
                    int cpu = (t1 == 1) ? comps[i].code : comps[j].code;
                    int mb  = (t1 == 2) ? comps[i].code : comps[j].code;
                    if (!((cpu == 101 && mb == 201) ||
                          (cpu == 102 && mb == 202))) ok = false;
                }
                if (ok) {
                    CompatRec r{};
                    r.compCode   = comps[i].code;
                    r.compatCode = comps[j].code;
                    k.write((char*)&r, sizeof(r));
                }
            }
    }
}
```

4.7 Файл ui.h

```
#pragma once
#include "common.h"

void mainMenu();
void menuRead();
void menuView();
void menuSort();
void menuSearch();
void menuAdd();
void menuDelete();
void menuEdit();

void printTypes();
void printComponents();
void printCompats();
```

4.8 Файл ui.cpp

Модуль пользовательского интерфейса. Для вывода таблиц используются UTF-8-осознанные функции `padLeft`/`padRight`, чтобы колонки с кириллицей выравнивались корректно.

```
#include "ui.h"
#include "list.h"
#include "file_io.h"
#include "special.h"

// ---------- печать таблиц (UTF-8 aware) ----------
static const int W_CODE = 6, W_TYPE = 6, W_MAN = 16, W_MODEL = 20,
                 W_PAR = 26, W_PRICE = 12, W_AVAIL = 10;

void printTypes() {
    if (!types) { std::cout << "Список типов пуст.\n"; return; }
    std::cout << padRight("Код", W_CODE) << padRight("Название", 30) << "\n";
    std::cout << std::string(36, '-') << "\n";
    for (Type* p = types; p; p = p->next)
        std::cout << padRight(std::to_string(p->code), W_CODE)
                  << padRight(p->name, 30) << "\n";
}

void printComponents() {
    if (!components) { std::cout << "Список комплектующих пуст.\n"; return; }

    std::cout << padRight("Код",     W_CODE)
              << padRight("Тип",     W_TYPE)
              << padRight("Фирма",   W_MAN)
              << padRight("Модель",  W_MODEL)
              << padRight("Параметры", W_PAR)
              << padLeft ("Цена",    W_PRICE)
              << "  "
              << padLeft ("Наличие", W_AVAIL)
              << "\n";
    std::cout << std::string(W_CODE + W_TYPE + W_MAN + W_MODEL + W_PAR + W_PRICE
+ 2 + W_AVAIL, '-') << "\n";

    for (Component* p = components; p; p = p->next) {
        std::cout << padRight(std::to_string(p->code),     W_CODE)
                  << padRight(std::to_string(p->typeCode), W_TYPE)
                  << padRight(p->manufacturer,             W_MAN)
                  << padRight(p->model,                    W_MODEL)
                  << padRight(p->params,                   W_PAR)
                  << padLeft (fmtPrice(p->price),          W_PRICE)
                  << "  "
                  << padLeft (std::to_string(p->available),W_AVAIL)
                  << "\n";
    }
}

void printCompats() {
    if (!compats) { std::cout << "Список совместимости пуст.\n"; return; }
    std::cout << padRight("Код 1", 10) << padRight("Код 2", 10) << "\n";
    std::cout << std::string(20, '-') << "\n";
    for (Compat* p = compats; p; p = p->next)
        std::cout << padRight(std::to_string(p->compCode),   10)
```

```
                  << padRight(std::to_string(p->compatCode), 10) << "\n";
}

// ---------- п.1 ----------
void menuRead() {
    clearAll();
           if    (!fileExists(FILE_TYPES)    ||    !fileExists(FILE_COMPONENTS)
|| !fileExists(FILE_COMPAT)) {
        std::cout << "Файлы не найдены. Создаю тестовые данные...\n";
        createSampleFiles();
    }
    readTypesFromFile();
    readComponentsFromFile();
    readCompatsFromFile();
    std::cout << "Данные загружены в динамические списки.\n";
}

// ---------- п.2 ----------
void menuView() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Просмотр списков ===\n"
                  << "1. Типы комплектующих\n"
                  << "2. Комплектующие\n"
                  << "3. Совместимость\n"
                  << "4. Все списки\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: printTypes(); break;
            case 2: printComponents(); break;
            case 3: printCompats(); break;
            case 4: printTypes(); std::cout << "\n"; printComponents();
                    std::cout << "\n"; printCompats(); break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- п.3 ----------
void menuSort() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Сортировка ===\n"
                  << "1. Типы по коду\n"
                  << "2. Типы по названию\n"
                  << "3. Комплектующие по коду\n"
                  << "4. Комплектующие по цене (возр.)\n"
                  << "5. Комплектующие по цене (убыв.)\n"
                  << "6. Комплектующие по коду типа\n"
                  << "7. Комплектующие по фирме\n"
                  << "8. Совместимость по коду 1\n"
                  << "9. Совместимость по коду 2\n"
```

```
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: sortTypes(1);      std::cout << "Отсортировано.\n"; break;
            case 2: sortTypes(2);      std::cout << "Отсортировано.\n"; break;
            case 3: sortComponents(1); std::cout << "Отсортировано.\n"; break;
            case 4: sortComponents(2); std::cout << "Отсортировано.\n"; break;
            case 5: sortComponents(3); std::cout << "Отсортировано.\n"; break;
            case 6: sortComponents(4); std::cout << "Отсортировано.\n"; break;
            case 7: sortComponents(5); std::cout << "Отсортировано.\n"; break;
            case 8: sortCompats(1);    std::cout << "Отсортировано.\n"; break;
            case 9: sortCompats(2);    std::cout << "Отсортировано.\n"; break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- п.4 ----------
static void searchTypes() {
    int code = inputInt("Код типа (-1 любой): ");
    std::cout << "Часть названия (пусто = любая): ";
    std::string nameFilter; std::getline(std::cin, nameFilter);

    bool found = false;
    for (Type* p = types; p; p = p->next) {
        if (code != -1 && p->code != code) continue;
           if  (!nameFilter.empty()  &&  std::string(p->name).find(nameFilter)  ==
std::string::npos) continue;
        std::cout << padRight(std::to_string(p->code), W_CODE) << p->name << "\n";
        found = true;
    }
    if (!found) std::cout << "Не найдено.\n";
}
static void searchComponents() {
    int tc = inputInt("Код типа (-1 любой): ");
    double mn = inputDouble("Мин. цена (-1 любая): ");
    double mx = inputDouble("Макс. цена (-1 любая): ");
    int only = inputInt("Только в наличии? 1-да, 0-нет: ");

    bool found = false;
    for (Component* p = components; p; p = p->next) {
        if (tc != -1 && p->typeCode != tc) continue;
        if (mn >= 0 && p->price < mn) continue;
        if (mx >= 0 && p->price > mx) continue;
        if (only == 1 && p->available <= 0) continue;
        std::cout << padRight(std::to_string(p->code), W_CODE)
                  << padRight(p->manufacturer, W_MAN)
                  << padRight(p->model, W_MODEL)
                  << padLeft(fmtPrice(p->price), W_PRICE) << "\n";
        found = true;
    }
    if (!found) std::cout << "Не найдено.\n";
}
static void searchCompatByComponent() {
    int code = inputInt("Код комплектующего: ");
```

```
    bool found = false;
    for (Compat* p = compats; p; p = p->next)
        if (p->compCode == code || p->compatCode == code) {
            std::cout << padRight(std::to_string(p->compCode), 10)
                      << padRight(std::to_string(p->compatCode), 10) << "\n";
            found = true;
        }
    if (!found) std::cout << "Не найдено.\n";
}
static void searchCompatibleByTypeAndComponent() {
    int tc = inputInt("Код типа комплектующего: ");
    int cc = inputInt("Код конкретного комплектующего: ");
    bool found = false;
    for (Component* p = components; p; p = p->next)
        if (p->typeCode == tc && areCompatible(cc, p->code)) {
            std::cout << padRight(std::to_string(p->code), W_CODE)
                      << padRight(p->manufacturer, W_MAN)
                      << padRight(p->model, W_MODEL) << "\n";
            found = true;
        }
    if (!found) std::cout << "Не найдено.\n";
}

void menuSearch() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Поиск ===\n"
                  << "1. Типы по фильтру\n"
                  << "2. Комплектующие по фильтру\n"
                  << "3. Совместимость по коду комплектующего\n"
                  << "4. Комплектующие заданного типа, совместимые с конкретным\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: searchTypes(); break;
            case 2: searchComponents(); break;
            case 3: searchCompatByComponent(); break;
            case 4: searchCompatibleByTypeAndComponent(); break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- п.5 ----------
static void addTypeInteractive() {
    int code = inputInt("Код типа: ");
    if (typeExists(code)) { std::cout << "Такой код уже есть.\n"; return; }
    char name[64]; inputString("Название: ", name, 64);
    addTypeNode(code, name);
    std::cout << "Тип добавлен.\n";
}
static void addComponentInteractive() {
    int code = inputInt("Код комплектующего: ");
    if (componentExists(code)) { std::cout << "Такой код уже есть.\n"; return; }
```

```
    int tc = inputInt("Код типа: ");
    if (!typeExists(tc)) { std::cout << "Тип не найден.\n"; return; }
    char man[64], model[64], par[128];
    inputString("Фирма: ", man, 64);
    inputString("Модель: ", model, 64);
    inputString("Параметры: ", par, 128);
    double price = inputDouble("Цена: ");
    int avail = inputInt("Наличие: ");
    addComponentNode(code, tc, man, model, par, price, avail);
    std::cout << "Комплектующее добавлено.\n";
}
static void addCompatInteractive() {
    int a = inputInt("Код комплектующего: ");
    int b = inputInt("Код совместимого: ");
     if (!componentExists(a) || !componentExists(b)) { std::cout << "Код не
найден.\n"; return; }
    addCompatNode(a, b);
    std::cout << "Совместимость добавлена.\n";
}

void menuAdd() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Добавление ===\n"
                  << "1. Тип комплектующего\n"
                  << "2. Комплектующее\n"
                  << "3. Совместимость\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: addTypeInteractive(); break;
            case 2: addComponentInteractive(); break;
            case 3: addCompatInteractive(); break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- п.6 ----------
static void deleteTypeInteractive() {
    int code = inputInt("Код типа для удаления: ");
    Type* p = types; Type* prev = nullptr;
    while (p) {
        if (p->code == code) {
            if (prev) prev->next = p->next; else types = p->next;
            delete p; std::cout << "Тип удалён.\n"; return;
        }
        prev = p; p = p->next;
    }
    std::cout << "Тип не найден.\n";
}
static void deleteComponentInteractive() {
    int code = inputInt("Код комплектующего для удаления: ");
    Component* p = components; Component* prev = nullptr;
```

```
    while (p) {
        if (p->code == code) {
            if (prev) prev->next = p->next; else components = p->next;
            delete p;
            removeCompatByComponentCode(code);
           std::cout << "Комплектующее удалено, связки совместимости очищены.\n";
            return;
        }
        prev = p; p = p->next;
    }
    std::cout << "Комплектующее не найдено.\n";
}
static void deleteCompatInteractive() {
    int a = inputInt("Код 1: ");
    int b = inputInt("Код 2: ");
    while (compats &&
          ((compats->compCode == a && compats->compatCode == b) ||
           (compats->compCode == b && compats->compatCode == a))) {
        Compat* p = compats; compats = compats->next; delete p;
    }
    Compat* p = compats;
    while (p && p->next) {
        bool m = (p->next->compCode == a && p->next->compatCode == b) ||
                 (p->next->compCode == b && p->next->compatCode == a);
        if (m) { Compat* d = p->next; p->next = d->next; delete d; }
        else p = p->next;
    }
    std::cout << "Запись совместимости удалена, если была.\n";
}

void menuDelete() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Удаление ===\n"
                  << "1. Тип комплектующего\n"
                  << "2. Комплектующее\n"
                  << "3. Совместимость\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: deleteTypeInteractive(); break;
            case 2: deleteComponentInteractive(); break;
            case 3: deleteCompatInteractive(); break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- п.7 ----------
static void editTypeInteractive() {
    int code = inputInt("Код типа для редактирования: ");
    for (Type* p = types; p; p = p->next)
        if (p->code == code) {
            char n[64]; inputString("Новое название: ", n, 64);
```

```
            safeCopy(p->name, n, 64);
            std::cout << "Тип отредактирован.\n"; return;
        }
    std::cout << "Тип не найден.\n";
}
static void editComponentInteractive() {
    int code = inputInt("Код комплектующего для редактирования: ");
    for (Component* p = components; p; p = p->next)
        if (p->code == code) {
            int tc = inputInt("Новый код типа: ");
            if (!typeExists(tc)) { std::cout << "Тип не найден.\n"; return; }
            char man[64], model[64], par[128];
            inputString("Новая фирма: ", man, 64);
            inputString("Новая модель: ", model, 64);
            inputString("Новые параметры: ", par, 128);
            double price = inputDouble("Новая цена: ");
            int avail = inputInt("Новое наличие: ");
            p->typeCode = tc;
            safeCopy(p->manufacturer, man, 64);
            safeCopy(p->model, model, 64);
            safeCopy(p->params, par, 128);
            p->price = price; p->available = avail;
            std::cout << "Комплектующее отредактировано.\n"; return;
        }
    std::cout << "Комплектующее не найдено.\n";
}
static void editCompatInteractive() {
    int a = inputInt("Старый код 1: ");
    int b = inputInt("Старый код 2: ");
    for (Compat* p = compats; p; p = p->next) {
        bool m = (p->compCode == a && p->compatCode == b) ||
                 (p->compCode == b && p->compatCode == a);
        if (m) {
            int na = inputInt("Новый код 1: ");
            int nb = inputInt("Новый код 2: ");
            if (!componentExists(na) || !componentExists(nb)) {
                std::cout << "Новые коды не найдены.\n"; return;
            }
            p->compCode = na; p->compatCode = nb;
            std::cout << "Совместимость отредактирована.\n"; return;
        }
    }
    std::cout << "Запись совместимости не найдена.\n";
}

void menuEdit() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Редактирование ===\n"
                  << "1. Тип комплектующего\n"
                  << "2. Комплектующее\n"
                  << "3. Совместимость\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: editTypeInteractive(); break;
```

```
            case 2: editComponentInteractive(); break;
            case 3: editCompatInteractive(); break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}

// ---------- главное меню ----------
void mainMenu() {
    int choice;
    do {
        clearScreen();
        std::cout << "=== Главное меню ===\n"
                  << " 1. Чтение данных из файла\n"
                  << " 2. Просмотр всего списка\n"
                  << " 3. Сортировка данных\n"
                  << " 4. Поиск данных\n"
                  << " 5. Добавление данных\n"
                  << " 6. Удаление данных\n"
                  << " 7. Редактирование данных\n"
                  << " 8. Подбор и оформление заказа\n"
                  << " 9. Выход без сохранения\n"
                  << "10. Выход с сохранением\n";
        choice = inputInt("Выбор: ");
        clearScreen();

        switch (choice) {
            case 1:
                menuRead();
                pauseScreen();
                break;
            case 2:
                menuView();
                break;
            case 3:
                menuSort();
                break;
            case 4:
                menuSearch();
                break;
            case 5:
                menuAdd();
                break;
            case 6:
                menuDelete();
                break;
            case 7:
                menuEdit();
                break;
            case 8:
                menuSpecial();
                break;
            case 9: {
                std::cout << "Внимание! Изменения не будут сохранены.\n"
                          << "Вы действительно хотите выйти? 1-да, 0-отмена: ";
                int confirm = inputInt("");
```

```
                if (confirm == 1) {
                    clearAll();
                    std::cout << "Выход без сохранения.\n";
                    return;
                }
                std::cout << "Выход отменён.\n";
                pauseScreen();
                break;
            }
            case 10:
                saveAllFiles();
                clearAll();
                std::cout << "Выход с сохранением.\n";
                return;
            default:
                std::cout << "Неверный пункт меню.\n";
                pauseScreen();
                break;
        }
    } while (true);
}
```

4.9 Файл special.h

```
#pragma once
#include "common.h"

void menuSpecial();
void resetLastBuilds();
```

4.10 Файл special.cpp

Модуль реализует две специальные функции: подбор вариантов комплектации ПК в заданном ценовом диапазоне и оформление заказа выбранного варианта.

```
#include "special.h"
#include "list.h"

struct Build { std::vector<Component*> comps; double total; };

static std::vector<Build> lastBuilds;

void resetLastBuilds() { lastBuilds.clear(); }

// Печать одного варианта в поток (экран или файл)
static void printBuild(std::ostream& os, const Build& b, int num) {
    os << "Вариант " << num
       << ", итого: " << fmtPrice(b.total) << " руб.\n";
    for (Component* c : b.comps) {
        os << "  " << padRight(std::to_string(c->code), 6)
           << padRight(std::to_string(c->typeCode), 6)
           << padRight(c->manufacturer, 16)
           << padRight(c->model, 20)
           << padRight(c->params, 26)
           << padLeft(fmtPrice(c->price), 12) << " руб."
```

```
            << "  наличие: " << c->available << "\n";
    }
    os << "----------------------------------------\n";
}

// СФ 1: подбор вариантов комплектации
static void buildVariants() {
    if (!types || !components) {
        std::cout << "Сначала загрузите данные (п.1).\n";
        return;
    }

    double mn = inputDouble("Мин. стоимость: ");
    double mx = inputDouble("Макс. стоимость: ");
    if (mn > mx) {
        std::cout << "Минимум больше максимума.\n";
        return;
    }

    // Список типов, для которых есть хотя бы одно комплектующее в наличии
    std::vector<int> typeCodes;
    for (Type* t = types; t; t = t->next)
        for (Component* c = components; c; c = c->next)
            if (c->typeCode == t->code && c->available > 0) {
                typeCodes.push_back(t->code);
                break;
            }
    if (typeCodes.empty()) {
        std::cout << "Нет доступных комплектующих.\n";
        return;
    }

    // Для каждого типа — набор комплектующих на выбор
    std::vector<std::vector<Component*>> options;
    for (int tc : typeCodes) {
        std::vector<Component*> v;
        for (Component* c = components; c; c = c->next)
            if (c->typeCode == tc && c->available > 0)
                v.push_back(c);
        options.push_back(v);
    }

    std::vector<Component*> cur;
    std::vector<Build> builds;

    // Поиск в глубину с проверкой попарной совместимости
    std::function<void(int,double)> dfs = [&](int idx, double total) {
        if (total > mx + 1e-9) return;
        if (idx == (int)options.size()) {
            if (total >= mn - 1e-9) builds.push_back({cur, total});
            return;
        }
        for (Component* c : options[idx]) {
            if (total + c->price > mx + 1e-9) continue;
            bool ok = true;
            for (Component* prev : cur)
                if (!areCompatible(prev->code, c->code)) { ok = false; break; }
            if (!ok) continue;
```

```
            cur.push_back(c);
            dfs(idx + 1, total + c->price);
            cur.pop_back();
        }
    };
    dfs(0, 0);

    // Сортировка вариантов по возрастанию цены
    std::sort(builds.begin(), builds.end(),
              [](const Build& a, const Build& b) { return a.total < b.total; });
    lastBuilds = builds;

    std::ofstream f("sf_builds.txt");
    if (builds.empty()) {
        std::cout << "Варианты не найдены.\n";
        f << "Варианты не найдены.\n";
    } else {
        std::cout << "Найдено вариантов: " << builds.size() << "\n";
        f << "Найдено вариантов: " << builds.size() << "\n";
        for (size_t i = 0; i < builds.size(); ++i) {
            printBuild(std::cout, builds[i], (int)i + 1);
            printBuild(f,         builds[i], (int)i + 1);
        }
    }
    std::cout << "\nРезультат записан в sf_builds.txt\n";
}

// СФ 2: оформление заказа выбранного варианта
static void placeOrder() {
    if (lastBuilds.empty()) {
        std::cout << "Сначала выполните подбор (п.1).\n";
        return;
    }
    int num = inputInt("Номер варианта: ");
    if (num < 1 || num > (int)lastBuilds.size()) {
        std::cout << "Неверный номер.\n"; return;
    }
    std::string customer;
    std::cout << "ФИО заказчика: ";
    std::getline(std::cin, customer);
    int qty = inputInt("Количество комплектов: ");
    if (qty <= 0) { std::cout << "Неверное количество.\n"; return; }

    Build& b = lastBuilds[num - 1];

    // Проверка наличия всех комплектующих
    for (Component* c : b.comps)
        if (c->available < qty) {
            std::cout << "Недостаточно комплектующего " << c->code
                      << " (в наличии " << c->available << ").\n";
            return;
        }

    // Дозапись заказа в файл и вывод на экран
    std::ofstream f("sf_order.txt", std::ios::app);
    auto writeOrder = [&](std::ostream& os) {
        os << "Заказ: " << customer
           << ", количество: " << qty
```

```
            << ", сумма за комплект: " << fmtPrice(b.total)
            << ", общая сумма: " << fmtPrice(b.total * qty) << "\n";
        for (Component* c : b.comps)
            os << "  " << c->code << " | "
               << c->manufacturer << " " << c->model
               << " x" << qty << "\n";
        os << "----------------------------------------\n";
    };
    writeOrder(std::cout);
    writeOrder(f);
    f.close();

    // Списание наличия
    for (Component* c : b.comps) c->available -= qty;
    std::cout << "Заказ оформлен и записан в sf_order.txt\n";
}

// Подменю специальных функций
void menuSpecial() {
    int ch;
    do {
        clearScreen();
        std::cout << "=== Подбор и оформление заказа ===\n"
                  << "1. Подобрать варианты комплектации в ценовом диапазоне\n"
                  << "2. Оформить заказ выбранного варианта\n"
                  << "0. Назад\n";
        ch = inputInt("Выбор: ");
        clearScreen();
        switch (ch) {
            case 1: buildVariants(); break;
            case 2: placeOrder();    break;
            case 0: break;
            default: std::cout << "Неверный пункт.\n"; break;
        }
        if (ch != 0) pauseScreen();
    } while (ch != 0);
}
```

4.11 Файл tests.h

```
#pragma once
void runAllTests();
```

4.12 Файл tests.cpp

Реализован собственный мини-фреймворк: макрос `CHECK(условие, сообщение)` увеличивает счётчик пройденных или проваленных тестов и печатает результат.

```
#include "tests.h"
#include "common.h"
#include "list.h"
#include "file_io.h"

static int passed = 0, failed = 0;

#define CHECK(cond, msg) do { \
    if (cond) { ++passed; std::cout << "[ OK ] " << msg << "\n"; } \
    else      { ++failed; std::cout << "[FAIL] " << msg << "\n"; } \
} while (0)

// ---------- UTF-8 утилиты ----------
static void testUtf8() {
    std::cout << "\n-- UTF-8 утилиты --\n";
    CHECK(utf8Length("abc") == 3,    "utf8Length: ascii");
    CHECK(utf8Length("привет") == 6, "utf8Length: кириллица");
    CHECK(padRight("аб", 5).size() == 5 + 2, "padRight: длина в байтах корректна");
    CHECK(utf8Length(padRight("аб", 5)) == 5, "padRight: длина в символах = 5");
    CHECK(utf8Length(padLeft ("аб", 5)) == 5, "padLeft: длина в символах = 5");
}

// ---------- Списки ----------
static void testLists() {
    std::cout << "\n-- Динамические списки --\n";
    clearAll();
    CHECK(types == nullptr, "clearAll: types пуст");

    addTypeNode(1, "A");
    addTypeNode(2, "B");
    CHECK(types && types->code == 1 && types->next->code == 2,
          "addTypeNode: порядок сохранён");
    CHECK(typeExists(1) && typeExists(2) && !typeExists(3),
          "typeExists: работает");

    addComponentNode(10, 1, "M1", "Md1", "p1", 100.0, 1);
    addComponentNode(20, 2, "M2", "Md2", "p2", 200.0, 2);
    CHECK(componentExists(10) && componentExists(20) && !componentExists(30),
          "componentExists: работает");

    addCompatNode(10, 20);
    CHECK(areCompatible(10, 20), "areCompatible: симметрия 10-20");
    CHECK(areCompatible(20, 10), "areCompatible: симметрия 20-10");
    CHECK(areCompatible(10, 10), "areCompatible: self");
    CHECK(!areCompatible(10, 99), "areCompatible: отсутствие");
```

```
    removeCompatByComponentCode(20);
    CHECK(!areCompatible(10, 20), "removeCompat: удаление");

    clearAll();
    CHECK(!types && !components && !compats, "clearAll: всё очищено");
}

// ---------- Сортировка ----------
static void testSort() {
    std::cout << "\n-- Сортировка --\n";
    clearAll();
    addComponentNode(3, 1, "C", "M", "p", 300, 1);
    addComponentNode(1, 1, "A", "M", "p", 100, 1);
    addComponentNode(2, 1, "B", "M", "p", 200, 1);

    sortComponents(2); // по цене возрастание
    CHECK(components->price == 100, "sort: цена 1-я = 100");
    CHECK(components->next->price == 200, "sort: цена 2-я = 200");
    CHECK(components->next->next->price == 300, "sort: цена 3-я = 300");

    sortComponents(3); // по цене убывание
    CHECK(components->price == 300, "sort: убыв. 1-я = 300");

    sortComponents(1); // по коду
    CHECK(components->code == 1, "sort: код 1-й = 1");
    CHECK(components->next->code == 2, "sort: код 2-й = 2");
    CHECK(components->next->next->code == 3, "sort: код 3-й = 3");
    clearAll();

    addTypeNode(5, "E");
    addTypeNode(1, "A");
    addTypeNode(3, "C");
    sortTypes(1);
    CHECK(types->code == 1 && types->next->code == 3
          && types->next->next->code == 5, "sortTypes: по коду");
    sortTypes(2);
    CHECK(std::string(types->name) == "A" &&
          std::string(types->next->name) == "C" &&
          std::string(types->next->next->name) == "E",
          "sortTypes: по названию");
    clearAll();
}

// ---------- Файлы: round-trip ----------
static void testFileRoundtrip() {
    std::cout << "\n-- Файлы: round-trip --\n";
    clearAll();

    addTypeNode(101, "TestType");
    addComponentNode(201, 101, "Man", "Mod", "Par", 1234.5, 7);
    addCompatNode(201, 201);
    saveAllFiles();

    // Перечитываем и проверяем
    clearAll();
    readTypesFromFile();
    readComponentsFromFile();
```

```
    readCompatsFromFile();

    CHECK(types && types->code == 101
          && std::string(types->name) == "TestType",
          "файл types: round-trip");
    CHECK(components && components->code == 201
          && components->typeCode == 101
          && components->price == 1234.5
          && components->available == 7
          && std::string(components->manufacturer) == "Man",
          "файл components: round-trip");
    CHECK(compats && compats->compCode == 201
          && compats->compatCode == 201,
          "файл compat: round-trip");

    clearAll();
    // Восстанавливаем демонстрационный набор
    createSampleFiles();
}

// ---------- Логика СФ ----------
static void testSpecialLogic() {
    std::cout << "\n-- Логика СФ --\n";
    clearAll();

    addTypeNode(1, "CPU");
    addTypeNode(2, "MB");
    addComponentNode(11, 1, "X", "x", "", 5000, 2);
    addComponentNode(12, 2, "Y", "y", "", 3000, 2);
    addCompatNode(11, 12);

    // Проверка: существует хотя бы одна пара совместимых
    // комплектующих в диапазоне [1000, 10000]
    int found = 0;
    for (Component* a = components; a; a = a->next)
        for (Component* b = components; b; b = b->next)
            if (a->typeCode != b->typeCode
                && areCompatible(a->code, b->code)
                && a->available > 0 && b->available > 0
                && a->price + b->price >= 1000
                && a->price + b->price <= 10000)
                ++found;

    CHECK(found >= 1, "СФ: найден как минимум 1 вариант");
    clearAll();
}

// Точка входа
void runAllTests() {
    std::cout << "====================\n";
    std::cout << " Запуск тестов\n";
    std::cout << "====================\n";

    testUtf8();
    testLists();
    testSort();
    testFileRoundtrip();
    testSpecialLogic();
```

```
    std::cout << "\n====================\n";
    std::cout << "Пройдено: " << passed << "\n";
    std::cout << "Провалено: " << failed << "\n";
    std::cout << "====================\n";

    // Восстанавливаем демонстрационные файлы
    createSampleFiles();
}
```

4.13 Файл main.cpp

```
#include "common.h"
#include "tests.h"
#include "ui.h"

#ifdef _WIN32
#include <windows.h>
#endif

int main(int argc, char** argv) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif

    // Режим запуска тестов: ./app --test
    if (argc > 1 && std::string(argv[1]) == "--test") {
        runAllTests();
        return 0;
    }

    mainMenu();
    return 0;
}
```

5 GIT-ЛОГИ

В ходе разработки использовалась система контроля версий. Таблица 6 сделана при помощи команды: *git log --no-merges --date=format:'%d.%m.%Y %H:%M:%S' --pretty=format:"%h%x09%an%x09%ad%x09%s" > git_log.txt*

Таблица 6 – Git-логи

Хеш	Автор	Дата	Сообщение
963ff9f	Aleksey Moiseenko	10.10.2026 10:32:20	Add .gitignore and sample query output
b422089	Aleksey Moiseenko	07.10.2026 21:42:15	Add built-in tests, entry point and Makefile
d49a4c1	Aleksey Moiseenko	07.10.2026 18:03:46	Add console menu UI and special queries (build selection, order)
ca6ee7d	Aleksey Moiseenko	07.10.2026 11:36:14	Add binary file storage with sample data generation
5beb4fd	Aleksey Moiseenko	05.10.2026 13:10:23	Add data structures, shared utilities and linked list operations

ЗАКЛЮЧЕНИЕ

В ходе учебной практики разработано программное средство «Система учёта комплектующих персональных компьютеров и подбора сборок», полностью соответствующее техническому заданию.

В процессе работы были решены следующие задачи:

- спроектирована модульная архитектура из семи модулей;
- реализованы все десять пунктов пользовательского меню, включая две специальные функции;
- использованы динамические односвязные списки с ручным управлением памятью;
- реализовано хранение данных в трёх типизированных бинарных файлах;
- разработаны модульные тесты;
- подготовлена техническая документация.

Программа успешно прошла отладку и тестирование. Все модульные тесты проходят без ошибок, ручные тест-кейсы дают ожидаемый результат.

Дальнейшее развитие системы может быть направлено на:

- добавление графического интерфейса;
- экспорт отчётов в формат CSV или XLSX;
- поддержку нескольких складов и ценовых политик;
- учёт скидок и акций при подборе сборки.